AGENCYOS

Security, Permissions, Audit & Data Governance

Document 19 of the AgencyOS Master Documentation Set

AUTHENTICATION • RBAC • RLS • ORGANIZATION ISOLATION • SECRETS • API SECURITY • AI PERMISSIONS • AUDIT LOGS • DATA RETENTION • BACKUPS

Purpose: Define the security and governance foundation that protects AgencyOS, its organizations, clients, projects, financial data, AI agents, credentials, integrations and historical records while enforcing least privilege, tenant isolation, traceability and recoverability.

1. Security Core Principles

• Security is enforced by the platform, not by AI-agent instructions alone.

• Every request is authenticated, authorized and scoped to the correct organization/project/resource.

• Least privilege is the default.

• Organization and client data must be isolated.

• Sensitive actions require stronger authorization and, where configured, approval.

• Secrets are never exposed to ordinary agents or client users.

• Every important security-sensitive action is auditable.

• Backups and recovery are part of security, not an afterthought.

• Security controls must fail closed rather than silently granting access.

2. Security Architecture

> IDENTITY → AUTHENTICATION → SESSION → ORGANIZATION CONTEXT → ROLE/PERMISSION CHECK → RESOURCE/RLS CHECK → POLICY CHECK → ACTION → AUDIT EVENT.

• Backend authorization is authoritative.

• Frontend visibility is not a security boundary.

• Direct database/API access must enforce the same tenant and permission rules.

• Internal services use authenticated service identities.

3. Authentication

• Secure login.

• Email verification where configured.

• Password policy.

• Password reset.

• Session expiration.

• Session revocation.

• Optional MFA.

• Device/session visibility where supported.

• Login attempt monitoring.

• Rate limiting.

• Suspicious-login detection where configured.

• Account lock/step-up verification policy.

• Secure logout.

4. Identity Model

• User.

• Organization.

• Organization membership.

• Role.

• Permission.

• Project membership.

• Client membership.

• Service/agent identity.

• API credential.

• Session.

• Each identity has a unique internal ID.

5. Multi-Organization / Tenant Isolation

• Every organization is a security boundary.

• Organization-owned records carry organization context.

• Users can belong to one or multiple organizations only when explicitly supported.

• Cross-organization access is denied by default.

• Admin actions must still respect organization boundaries unless a global platform-admin role explicitly permits cross-tenant operations.

• Background jobs must preserve tenant context.

• Exports, files, logs and search indexes must preserve tenant isolation.

6. Row-Level Security (RLS)

• Database-level RLS should enforce organization/resource access where supported by the database architecture.

• Queries must not rely solely on application filters.

• Project/client records should be scoped to authorized organization/project membership.

• Service roles are tightly controlled.

• RLS policies must be tested for allow and deny cases.

• Any privileged database path must have explicit justification and auditability.

7. Resource-Level Authorization

• Organization access.

• Project access.

• Client access.

• Finance access.

• Design access.

• Development access.

• QA access.

• Deployment access.

• Secrets access.

• Admin configuration access.

• Each resource type has explicit permissions.

• Having access to one project must not imply access to another.

8. Role-Based Access Control

• Platform Admin.

• Organization Admin.

• Finance/Admin Finance.

• Project Manager.

• Sales Agent.

• Designer.

• Developer.

• QA.

• Customer Success.

• Support.

• Client Admin.

• Client Viewer.

• AI Agent/service roles.

• Exact role names are configurable, but permissions should remain explicit.

9. Permission Model

• Use action-based permissions such as VIEW, CREATE, EDIT, APPROVE, DELETE, EXPORT, DEPLOY, REFUND, VERIFY_PAYMENT, MANAGE_SECRETS and MANAGE_USERS.

• Sensitive actions require explicit permission.

• Permissions are checked server-side.

• Role changes are audited.

• Permission inheritance must be understandable.

• Deny-by-default for undefined actions.

10. Approval & Separation of Duties

• High-risk actions should not be self-approved by the same agent/user where policy requires separation.

• Examples: refunds, production deployment, secret rotation, organization deletion, major permission changes, financial overrides and release exceptions.

• Approval records include actor, approver, timestamp, object/version and reason.

• Emergency overrides are time-bound/audited where supported.

11. AI Agent Identity & Permissions

• Every autonomous agent has a distinct identity.

• Agents receive scoped permissions rather than broad Admin access.

• Agent permissions are limited by organization, project, task and action.

• Agents can only access tools necessary for their assigned role.

• AI model choice does not automatically grant additional permissions.

• Agent-to-agent calls are authenticated and authorized.

• Agent permissions can be revoked centrally.

12. AI Permission Examples

• Sales Agent: CRM/lead data, approved pricing policies and communication tools; no secret-store or production-deploy access.

• PM Agent: project/task/scope data; controlled client communication; no unrestricted financial override.

• Finance Agent: invoices/payment records and verification workflow; no source-code deployment access.

• Developer Agent: assigned repository/project/build tools; no unrestricted client financial data.

• QA Agent: test/build environments and evidence; no production secret retrieval unless explicitly required.

• Deployment Agent: approved release artifact and deployment tool; no arbitrary source modification.

• Customer Success Agent: client/project/support context; no privileged credentials.

13. Tool-Level Security

• Each tool/API endpoint declares required permissions.

• Tool calls include authenticated identity and organization/project context.

• High-risk tools require policy checks.

• Tool arguments are validated.

• Tool outputs are filtered to authorized data.

• Agents cannot call hidden/internal tools merely by guessing names.

• Failed authorization attempts are auditable.

14. Secrets Management

• Store API keys, database credentials, OAuth secrets, webhook secrets and deployment credentials in a dedicated secret-management mechanism.

• Do not store secrets in ordinary database tables or project notes unless specifically protected.

• Never expose raw secrets to clients.

• Never place secrets in prompts, logs or generated source code.

• Use scoped credentials.

• Support rotation/revocation.

• Track secret metadata without exposing secret values.

• Separate development/staging/production credentials.

15. AgencyOS AI API Key Management

• Admin can configure approved AI providers and API credentials through a protected Admin page.

• API keys are encrypted/protected at rest.

• Only authorized backend services can use them.

• Agents receive provider access through controlled routing, not raw key visibility.

• Admin can activate/deactivate/rotate keys.

• Provider/model usage can be tracked without exposing the key.

• Failed provider authentication should trigger routing/fallback policy, not reveal the credential.

16. API Security

• Authentication on protected endpoints.

• Authorization on every protected action.

• Input/schema validation.

• Output filtering.

• Rate limiting.

• Request size limits.

• Idempotency for sensitive operations where applicable.

• Replay protection where required.

• Secure error handling.

• API versioning.

• Audit logging for sensitive endpoints.

• No sensitive data in URLs when avoidable.

17. Webhook Security

• Authenticate/verify webhook signatures where supported.

• Reject invalid or replayed events.

• Use idempotency keys.

• Restrict accepted event types.

• Log verification outcomes.

• Do not trust arbitrary webhook payloads as financial or project truth without validation.

18. File Security

• Project-scoped file authorization.

• Signed/temporary access where appropriate.

• File type/size restrictions.

• Malware/security scanning where available.

• Access logs for sensitive files.

• Do not expose private files through predictable URLs.

• Delete/revoke access according to retention policy.

• Separate client-visible files from internal files.

19. Data Classification

• Public.

• Internal.

• Confidential.

• Sensitive.

• Highly restricted/secret.

• Classification should determine access, storage, logging, retention and export rules.

• Examples include client contracts, financial records, personal contact data, credentials and security findings.

20. Data Minimization

• Collect only information required for the business workflow.

• Do not copy the same sensitive data into multiple systems unnecessarily.

• Do not retain raw credentials when a token/reference is sufficient.

• Do not expose internal agent context to clients.

• Limit AI context to the minimum required for the task.

21. Audit Logs

• Authentication events.

• Authorization failures.

• Role/permission changes.

• Organization membership changes.

• Secret configuration changes.

- API key/provider changes.
- Financial overrides.
- Payment verification.
- Refunds.
- Scope approvals.
- Production deployments.
- Rollbacks.
- Data exports.
- Deletion actions.
- AI agent privileged actions.
- Admin overrides.

22. Audit Event Schema

- Event ID.
- Timestamp.
- Actor ID.
- Actor type.
- Organization ID.
- Project ID where applicable.
- Action.
- Resource type/ID.
- Previous state summary where safe.
- New state summary where safe.
- Reason.
- Request/correlation ID.
- IP/device metadata where policy permits.
- Result: success/failure.
- Audit records must avoid storing raw secrets.

23. Immutable Audit History

- Ordinary users and agents cannot edit audit events.
- Audit events should be append-oriented.
- Administrative access to audit logs is itself audited.
- Retention should protect logs long enough for operational/security needs.
- Exported audit logs should preserve integrity and context.

24. Security Monitoring

- Repeated failed logins.
- Privilege escalation attempts.
- Unusual organization access.
- Repeated authorization failures.
- Secret-access anomalies.
- Large data exports.

• Unexpected deployment actions.

• Suspicious API traffic.

• Abnormal agent tool usage.

• Security monitoring should produce actionable alerts rather than raw noise.

25. Data Retention

• Define retention by data category.

• Financial records follow applicable business/legal requirements.

• Audit logs follow security/compliance policy.

• Project artifacts follow contract/business policy.

• Client communication follows configured retention.

• Temporary files have shorter retention.

• Expired records should be archived or deleted according to policy.

• Retention rules must be configurable and versioned.

26. Data Deletion

• Support authorized deletion workflows.

• Deletion requires permission and may require approval.

• Legal/financial retention requirements can prevent immediate deletion.

• Soft-delete can preserve operational recovery where appropriate.

• Permanent deletion should be explicit and audited.

• Deleting an organization must account for projects, files, users, financial records and audit requirements.

27. Backups

• Automated backups according to configured schedule.

• Database backups.

• Critical file/object backups.

• Configuration backups where appropriate.

• Backup encryption.

• Access-controlled backup storage.

• Backup retention policy.

• Backup integrity checks.

• Backup monitoring.

28. Disaster Recovery

• Define Recovery Point Objective (RPO).

• Define Recovery Time Objective (RTO).

• Document restore procedures.

• Test restores periodically.

• Maintain recovery dependencies.

• Keep emergency access procedures.

• Record recovery incidents.

• Do not assume a backup is usable without restore testing.

29. Business Continuity

• Identify critical AgencyOS functions.

• Prioritize CRM, project data, finance, communication, authentication and deployment systems.

• Define degraded-mode behavior.

• Define manual fallback procedures where necessary.

• Communicate major outages to affected users according to policy.

30. Encryption

• Encrypt sensitive data in transit.

• Encrypt sensitive data at rest where supported/required.

• Protect encryption keys separately from encrypted data.

• Rotate keys according to policy.

• Do not log plaintext sensitive data.

31. Environment Isolation

• Development, staging and production are separate security environments.

• Production credentials are not reused in development.

• Production data is not copied into development without approved protection/anonymization.

• Deployment permissions are environment-specific.

• Test agents cannot automatically access production secrets.

32. Database Security

• Least-privilege database roles.

• RLS policies.

• Encrypted connections.

• Migration controls.

• Backup/restore protection.

• Sensitive-column protection where applicable.

• Query auditing for high-risk operations.

• Prevent destructive operations outside authorized migration/admin workflows.

33. Data Export Security

• Export requires permission.

• Exports are scoped to organization/project.

• Sensitive fields can be masked.

• Export events are audited.

• Large exports may require approval.

• Download links expire where appropriate.

• Do not allow cross-tenant exports.

34. Client Data Isolation

• Clients can only access their organization/project records.

• Client users cannot access internal agency notes.
• Client users cannot access other clients.
• Client financial visibility is limited to their own authorized records.
• Client files are project-scoped.
• Client API access, if provided, uses scoped tokens.

35. Search & AI Context Isolation

• Search indexes must enforce tenant/project permissions.
• AI retrieval must apply the same authorization rules as the source data.
• An agent working on Project A must not retrieve Project B's private data.
• Shared memory contains only intentionally shared context.
• Client-facing AI must not retrieve internal agency notes or other-client data.

36. AI Memory Governance

• Classify memory as project, organization, client, agent-private or global policy.
• Only authorized memory is retrieved.
• Sensitive memory has restricted access.
• Memory deletion follows retention policy.
• Agent memory cannot become a hidden permission bypass.
• All high-risk memory access should be traceable.

37. AI Model Provider Security

• Approved providers/models are configured by Admin.
• Provider credentials are centrally managed.
• Routing policies restrict which agents can use which models.
• Sensitive data may require provider-specific restrictions.
• Provider requests should use minimum necessary context.
• Provider responses must not be treated as trusted instructions.
• Fallback providers inherit the same data/permission policy.

38. Prompt / Tool Injection Defense

• Treat external text as untrusted input.
• Do not let client messages redefine system permissions.
• Do not let retrieved documents authorize privileged actions.
• Require policy checks before tool execution.
• Separate data from instructions.
• Sensitive actions require structured authorization, not natural-language approval.

39. Security for Autonomous Agents

• Bound agent execution by task, time, organization, project and tools.
• Use maximum action limits where appropriate.
• Require confirmation/approval for high-risk actions.
• Stop agent execution after repeated policy violations.

• Record privileged tool calls.

• Allow Admin to revoke an agent's access immediately.

40. Security Incident Workflow

DETECT → TRIAGE → CONTAIN → INVESTIGATE → ERADICATE → RECOVER → VALIDATE → NOTIFY IF REQUIRED → POST-INCIDENT REVIEW.

• Incident records contain severity, affected scope, evidence, actions, owner and resolution.

• Security incidents can pause deployment or project completion.

41. Vulnerability Management

• Track dependency vulnerabilities.

• Track application security findings.

• Track infrastructure findings where applicable.

• Assign severity and priority.

• Set remediation deadlines.

• Retest fixes.

• Critical findings can block production according to policy.

42. Secure Development

• Code review.

• Static analysis.

• Dependency scanning.

• Secret scanning.

• Input validation.

• Authorization tests.

• Security-focused regression tests.

• Safe error handling.

• Secure configuration.

• Developer agents must follow the same security gates as human developers.

43. Security Testing of Permissions

• Test every role.

• Test cross-organization access.

• Test cross-project access.

• Test client-vs-internal access.

• Test denied actions.

• Test privilege escalation attempts.

• Test revoked permissions.

• Test expired sessions/tokens.

• Test service-agent boundaries.

44. Admin Security Controls

- Manage users/roles.
- Manage organization membership.
- Manage permissions.
- Manage API keys/provider credentials through secure interfaces.
- Configure retention.
- View audit logs.
- Manage security policies.
- Revoke sessions.
- Disable agents.
- Rotate credentials.
- Trigger backup/restore workflows where supported.
- Lock emergency operations.

45. Security Dashboard

- Authentication alerts.
- Authorization failures.
- Active privileged sessions.
- Role changes.
- Secret/API-key changes.
- Agent privileged actions.
- Security findings.
- Vulnerability status.
- Backup status.
- Restore-test status.
- Audit activity.
- Data-retention status.
- Incidents.

46. Security KPIs

- Failed-login rate.
- Authorization-denial rate.
- Critical vulnerabilities open.
- Time to remediate.
- Privileged-action volume.
- Secret rotations.
- Backup success rate.
- Restore-test success.
- Audit-log coverage.
- Security incidents.
- Cross-tenant access violations.
- These metrics should come from authoritative system logs.

47. Security Exceptions

• Exception ID.
• Control being bypassed.
• Reason.
• Risk assessment.
• Mitigation.
• Owner.
• Approver.
• Start/end date.
• Affected organization/project.
• Audit evidence.
• Exceptions expire automatically where possible.

48. Security Acceptance Criteria

• Authentication is secure and configurable.
• Authorization is enforced server-side.
• RBAC exists.
• RLS/tenant isolation is enforced where applicable.
• Cross-organization access is denied by default.
• Client/project isolation is enforced.
• AI agents have scoped identities and permissions.
• Secrets are centrally protected.
• AI provider API keys are never exposed to agents unnecessarily.
• API security controls exist.
• Sensitive file access is protected.
• Audit logs cover privileged/security-sensitive actions.
• Audit records cannot be casually edited.
• Data retention is policy-driven.
• Deletion is controlled and auditable.
• Backups are encrypted and monitored.
• Restore procedures are tested.
• Development/staging/production are isolated.
• Security incidents and exceptions are tracked.
• Security cannot be bypassed through prompts or natural-language instructions.

49. Final Security Architecture

USER/AGENT IDENTITY → AUTHENTICATION → ORGANIZATION CONTEXT → RBAC → RLS/RESOURCE AUTHORIZATION → POLICY ENGINE → TOOL/API ACTION → AUDIT LOG → MONITORING → BACKUP/RECOVERY

For AI workflows: AGENT IDENTITY → SCOPED PERMISSIONS → APPROVED TOOLS → MINIMUM NECESSARY CONTEXT → POLICY CHECK → ACTION → AUDIT → REVOCATION/ROTATION WHEN REQUIRED.